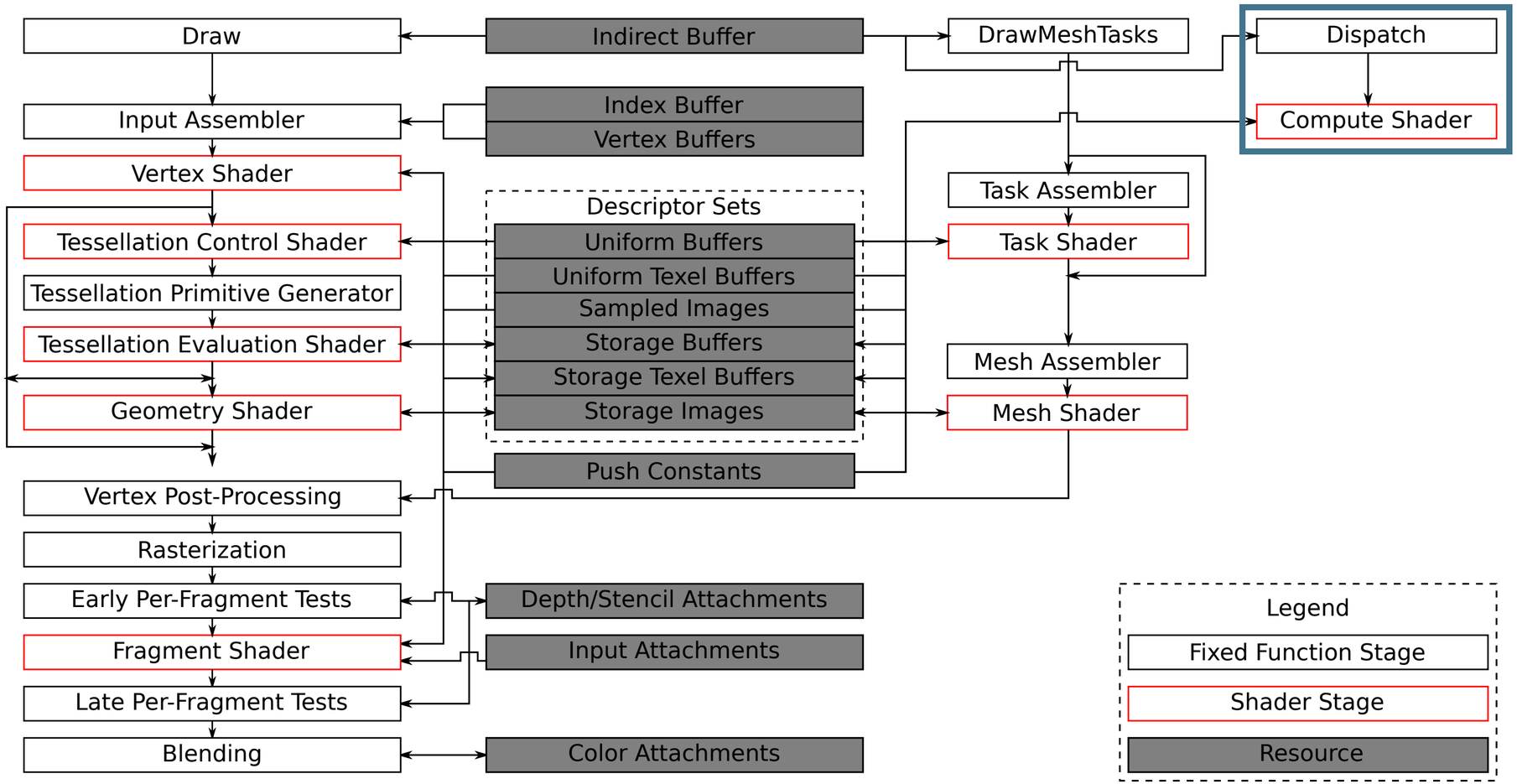
Crat3r
#02 - Compute Shaders, Descriptors, Buffers and Memory

Erik Eismann, Marc Stamminger

October 20, 2026

Vulkan Pipelines Overview

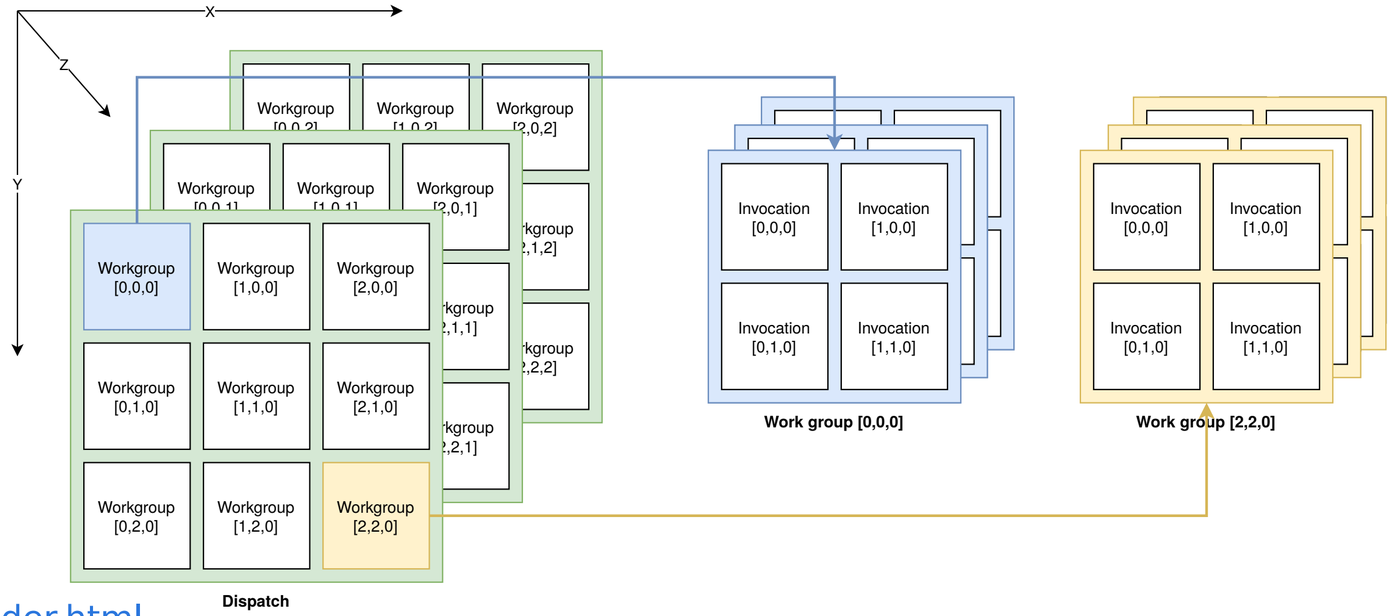


https://docs.vulkan.org/spec/latest/chapters/pipelines.html

Compute Shader Recap

Recap from Interactive Computer Graphics:

- Highly parallel shader invocation outside the rendering pipeline
- No user-defined inputs or outputs
- Typically reads/writes data from/to images or buffers
- 3D compute space in two levels: work groups and invocations
- Within one work group: fast shared memory
- Between work groups: slow global memory



https://docs.vulkan.org/tutorial/latest/11_Compute_Shader.html

Compute Shader Recap

```
#version 450
layout (local_size_x = 256, local_size_y = 1, local_size_z = 1) in;

layout (binding = 0) uniform ParameterUBO {
    float deltaTime;
    uint particleCount;
} ubo;
layout(std140, binding = 1) readonly buffer ParticleSSBOIn {
   Particle particlesIn[];
};
layout(std140, binding = 2) buffer ParticleSSBOOut {
   Particle particlesOut[];
};

void main() {
    uint idx = gl_GlobalInvocationID.x;
    if (idx >= ubo.particleCount) return;

    Particle particleIn = particlesIn[idx];
    particlesOut[idx].position = particleIn.position + particleIn.velocity.xy * ubo.deltaTime;
    particlesOut[idx].velocity = particleIn.velocity;
}
```

Compute Shader Dispatch

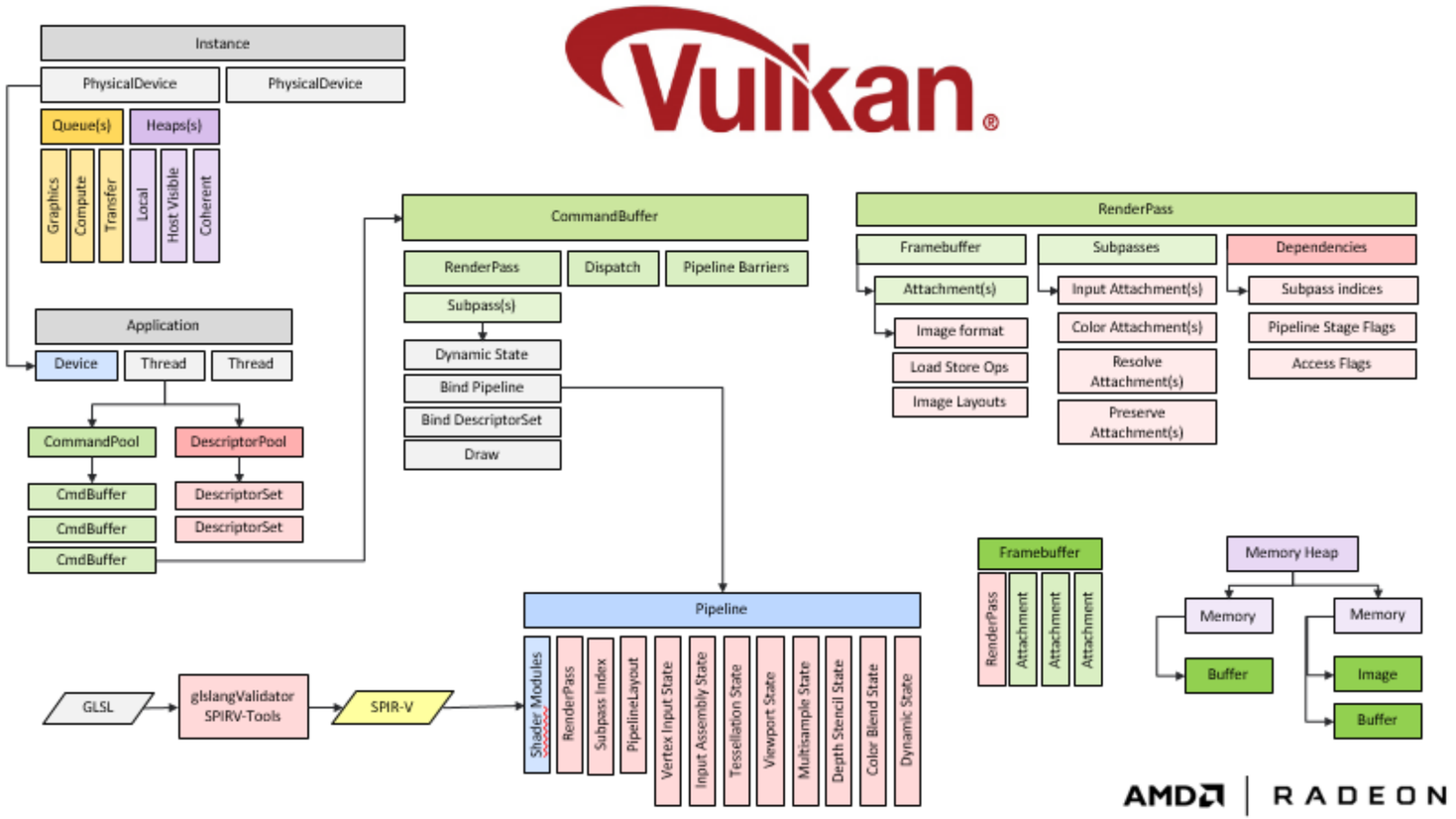
```
1 void vkCmdDispatch(
2     VkCommandBuffer   commandBuffer,
3     uint32_t          groupCountX,
4     uint32_t          groupCountY,
5     uint32_t          groupCountZ);
```

When the command is executed, a global workgroup consisting of `groupCountX` × `groupCountY` × `groupCountZ` local workgroups is assembled.

But where do we specify which compute shader to call?

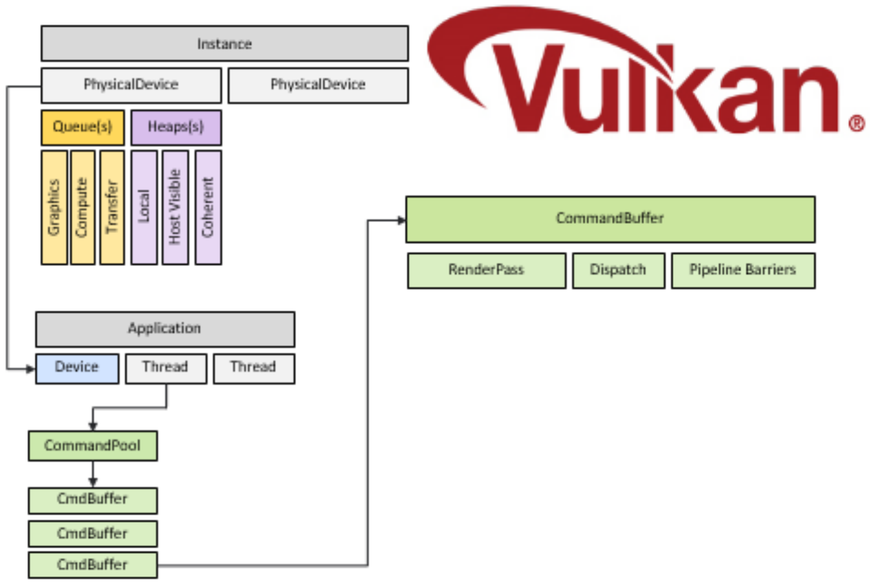
And what even is a command buffer?

Vulkan Architecture



https://gpuopen-librariesandsdks.github.io/V-EZ

Vulkan Architecture - Instance and Devices



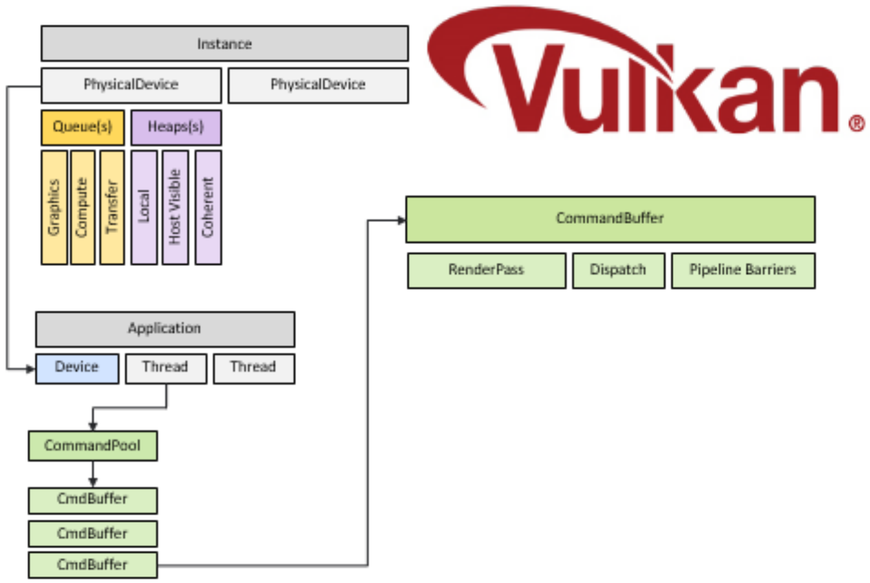
- **VkInstance**
  - Primary connection between app and Vulkan
  - Created with details about app and enabled layers / instance extensions
- **VkPhysicalDevice**
  - Represents actual GPU (or software implementation)
  - Enumeration of available devices through instance
  - Used to query GPU properties / features
- **VkDevice**
  - Logical device handle to a chosen GPU
  - First argument when creating almost any object.

AMD | RADEON

https://gpuopen-librariesandsdks.github.io/V-EZ

For now we'll assume these just exist. More details in later lecture.

Vulkan Architecture - Queues and Command Buffers



- **VkQueue**
  - Where work is submitted to
  - Created and destroyed implicitly with the device Retrieved from the device after initialization
  - Belongs to one queue family. Queue families describe a set of queues with certain capabilities (graphics, compute, transfer)
- **VkCommandBuffer**
  - Used to record commands (e.g. `vkCmdDraw`, `vkCmdDispatch`) before submitting to a queue
  - Allocated from a `VkCommandPool` that manages the memory used for recorded commands

AMD | RADEON

https://gpuopen-librariesandsdks.github.io/V-EZ

For now we'll assume these just exist. More details in later lecture.

Recording a Command Buffer

```
1  void recordCommandBuffer(VkCommandBuffer commandBuffer) {
2      // dispatch a compute shader here
3      ...
4
5
6
7
8
9
10
11
12
13
14
15
16 }
```

Compute Shader Dispatch

```
1 void vkCmdDispatch(
2     VkCommandBuffer   commandBuffer,
3     uint32_t          groupCountX,
4     uint32_t          groupCountY,
5     uint32_t          groupCountZ);
```

When the command is executed, a global workgroup consisting of `groupCountX` × `groupCountY` × `groupCountZ` *local workgroups is assembled.*

But where do we specify which compute shader to call?

VUID-vkCmdDispatch-None-08606
If the shaderObject feature is not enabled, a valid pipeline must be ***bound to the pipeline bind point*** *used by this command*

Binding a Pipeline

```
1 void vkCmdBindPipeline(
2     VkCommandBuffer        commandBuffer,
3     VkPipelineBindPoint    pipelineBindPoint,
4     VkPipeline             pipeline);
```

- Pipelines are bound to a command buffer using `vkCmdBindPipeline`.
- Affects all further commands using that bind point of the command buffer.
- Graphics, compute and ray-tracing pipelines all use the same `VkPipeline` handle, but different bind points (`VK_PIPELINE_BIND_POINT_GRAPHICS`, `VK_PIPELINE_BIND_POINT_COMPUTE`, …).
- Each bind point affects different calls (`vkCmdDraw`, `vkCmdDispatch`, `vkCmdTraceRaysKHR`, …).

Why have binding be stateful instead of passing pipelines to draw calls?

Switching pipelines is expensive on the GPU.
The decision when to switch is up to the application.

Recording a Command Buffer

```
1  void recordCommandBuffer(VkCommandBuffer commandBuffer) {
2      vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_COMPUTE, m_pipeline);
3
4      vkCmdDispatch(commandBuffer, div_ceil(NUM_PARTICLES, 256), 1, 1);
5
6      ...
7
8
9
10
11
12
13
14
15
16 }
```

Compute Pipeline Setup

```
1 VkResult vkCreateComputePipelines(
2     VkDevice                                    device,
3     VkPipelineCache                             pipelineCache,
4     uint32_t                                    createInfoCount,
5     const VkComputePipelineCreateInfo*          pCreateInfos,
6     const VkAllocationCallbacks*                pAllocator,
7     VkPipeline*                                 pPipelines);
```

```
1 typedef struct VkComputePipelineCreateInfo {
2     VkStructureType                             sType;
3     const void*                                 pNext;
4     VkPipelineCreateFlags                       flags;
5     VkPipelineShaderStageCreateInfo             stage;
6     VkPipelineLayout                            layout;
7     VkPipeline                                  basePipelineHandle;
8     int32_t                                     basePipelineIndex;
9 } VkComputePipelineCreateInfo;
```

- Expensive: the driver compiles SPIR-V to the GPU’s native ISA.
- Shaders are optimized specifically for the pipeline’s settings (more important for rasterization).
- Pipeline creation is batched to allow the driver to use parallel compilation and reuse resources.
- Pipeline caches speed up compilation of identical pipelines.
- Caches can be saved to disk for future runs.

Compute Pipeline Setup

```
typedef struct VkPipelineShaderStageCreateInfo {
    VkStructureType                     sType;
    const void*                         pNext;
    VkPipelineShaderStageCreateFlags    flags;
    VkShaderStageFlagBits               stage;
    VkShaderModule                      module;
    const char*                         pName;
    const VkSpecializationInfo*         pSpecializationInfo;
} VkPipelineShaderStageCreateInfo;
```

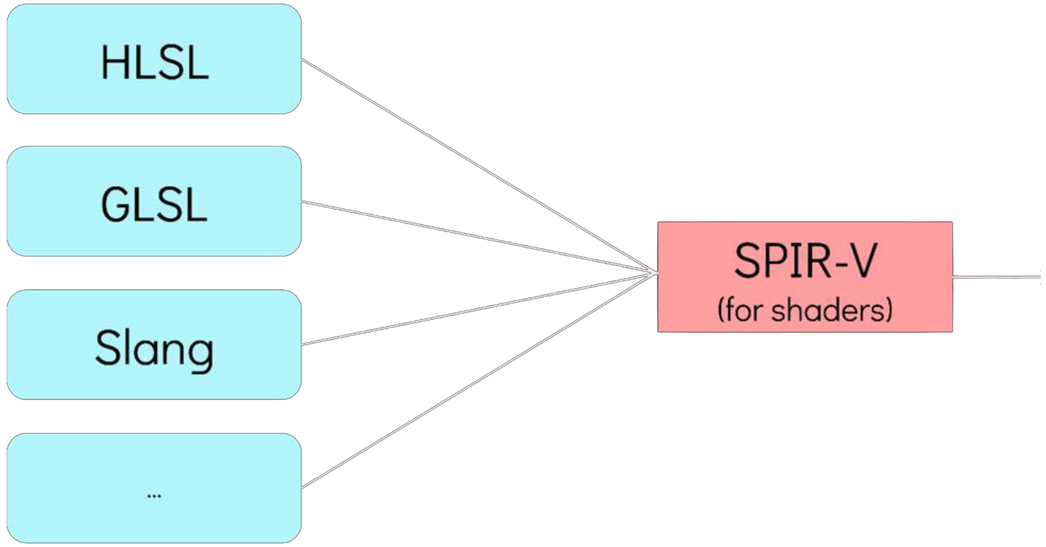
```
typedef enum VkShaderStageFlagBits {
    VK_SHADER_STAGE_VERTEX_BIT = 0x00000001,
    VK_SHADER_STAGE_TESSELLATION_CONTROL_BIT = 0x00000002,
    VK_SHADER_STAGE_TESSELLATION_EVALUATION_BIT = 0x00000004,
    VK_SHADER_STAGE_GEOMETRY_BIT = 0x00000008,
    VK_SHADER_STAGE_FRAGMENT_BIT = 0x00000010,
    VK_SHADER_STAGE_COMPUTE_BIT = 0x00000020,
    VK_SHADER_STAGE_ALL_GRAPHICS = 0x0000001F,
    ...
  } VkShaderStageFlagBits;
```

- Only one `VkShaderStageFlagBits` may be specified.
- Either specify a `VkShaderModule` or chain a `VkShaderModuleCreateInfo` (Vulkan 1.4+).
- `pName` is the name of the entry function.
- `VkSpecializationInfo` contains the specialization constants.

Compute Pipeline Setup

```
1 typedef struct VkShaderModuleCreateInfo {
2     VkStructureType              sType;
3     const void*                  pNext;
4     VkShaderModuleCreateFlags    flags;
5     size_t                       codeSize;
6     const uint32_t*              pCode;
7 } VkShaderModuleCreateInfo;
```

- Typically, `pCode` is a pointer to SPIR-V code.
- Passing invalid SPIR-V is undefined behavior.
- The original language of the shader is irrelevant at this point.



Specialization Constants

```
1 #version 450
2
3 layout (constant_id = 0) const uint light_count = 0U;
4 layout (constant_id = 1) const bool debug_out = false;
5
6 // special case for work group size
7 layout (local_size_x_id = 10, local_size_x = 256) in;
```

- Used to set constant values at pipeline creation
- Common use cases:
  - Eliminating entire branches behind always-false tests
  - Unrolling loops when the iteration count is known to be small
  - Setting work group size based on device limits
  - Sizing shared memory arrays
  - ...

Specialization Constants

```
1  struct SpecData {
2      uint32_t light_count;
3      VkBool32 debug_out;
4  };
5  SpecData data { 4, VK_TRUE };
6
7  VkSpecializationMapEntry entries[] = {
8      { 0, offsetof(SpecData, light_count), sizeof(uint32_t) },
9      { 1, offsetof(SpecData, debug_out),   sizeof(VkBool32) },
10 };
11
12 VkSpecializationInfo spec_info {
13     .mapEntryCount = sizeof(entries) / sizeof(entries[0]),
14     .pMapEntries   = entries,
15     .dataSize      = sizeof(data),
16     .pData         = &data,
17 };
```

- Unspecified constants resolve to their default values from the shader.

Compute Pipeline Setup - Recap

- Shader modules are created from SPIR-V code using `VkShaderModuleCreateInfo`.
- `VkPipelineShaderStageCreateInfo` adds the stage flag, entry function name and specialization constants to shader module.
- `VkComputePipelineCreateInfo` takes both the shader stage info and a `VkPipelineLayout`.
- Multiple pipelines can be created at once using `vkCreateComputePipelines`.

Recording a Command Buffer

```
1  void recordCommandBuffer(VkCommandBuffer commandBuffer) {
2      vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_COMPUTE, m_pipeline);
3
4      vkCmdDispatch(commandBuffer, div_ceil(NUM_PARTICLES, 256), 1, 1);
5
6      ...
7
8
9
10
11
12
13
14
15
16 }
```

Choosing a Work Group Size

```
1 layout (local_size_x = 256, local_size_y = 1, local_size_z = 1) in;
```

- GPUs typically run threads in lockstep groups of 32 or 64
- Use a multiple of 64 threads, e.g. 256×1×1 or 16×16×1
- Too large: more register and memory usage per work group
- Too small: less data reuse via shared memory, wasted lanes
- 256 invocations is typically a good starting point
- Work group count then is `(sz + wg - 1) / wg`
- Don’t forget bounds check in the shader

Important: Keep device limits in mind!

Physical Device Properties

```
1  void vkGetPhysicalDeviceProperties(
2      VkPhysicalDevice                    physicalDevice,
3      VkPhysicalDeviceProperties*         pProperties);
```

```
1  typedef struct VkPhysicalDeviceProperties {
2      uint32_t                            apiVersion;
3      uint32_t                            driverVersion;
4      uint32_t                            vendorID;
5      uint32_t                            deviceID;
6      VkPhysicalDeviceType                deviceType;
7      char                                deviceName[VK_MAX_PHYSICAL_DEVICE_NAME_SIZE];
8      uint8_t                             pipelineCacheUUID[VK_UUID_SIZE];
9      VkPhysicalDeviceLimits              limits;
10     VkPhysicalDeviceSparseProperties    sparseProperties;
11 } VkPhysicalDeviceProperties;
```

Physical Device Limits

```
typedef struct VkPhysicalDeviceLimits {
    uint32_t              maxImageDimension1D;
    uint32_t              maxImageDimension2D;
    uint32_t              maxImageDimension3D;
    uint32_t              maxImageDimensionCube;
    uint32_t              maxImageArrayLayers;
    uint32_t              maxTexelBufferElements;
    uint32_t              maxUniformBufferRange;
    uint32_t              maxStorageBufferRange;
    uint32_t              maxPushConstantsSize;
    uint32_t              maxMemoryAllocationCount;
    uint32_t              maxSamplerAllocationCount;
    VkDeviceSize          bufferImageGranularity;
    VkDeviceSize          sparseAddressSpaceSize;
    uint32_t              maxBoundDescriptorSets;
    uint32_t              maxPerStageDescriptorSamplers;
    uint32_t              maxPerStageDescriptorUniformBuffers;
    uint32_t              maxPerStageDescriptorStorageBuffers;
    uint32_t              maxPerStageDescriptorSampledImages;
    uint32_t              maxPerStageDescriptorStorageImages;
    uint32_t              maxPerStageDescriptorInputAttachments;
    uint32_t              maxPerStageResources;
    uint32_t              maxDescriptorSetSamplers;
```

Work Group Device Limits

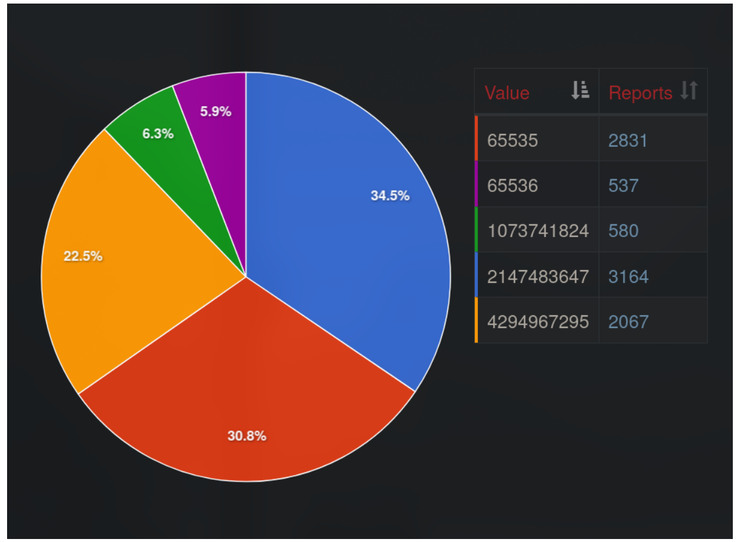
- `maxComputeWorkGroupCount[d]` is the maximum number of local workgroups that can be dispatched by a single dispatching command in dimension `d`.

- `maxComputeWorkGroupInvocations` is the maximum total number of compute shader invocations in a single local workgroup (`x×y×z`).

- `maxComputeWorkGroupSize[d]` is the maximum size of a local compute workgroup in dimension `d`.

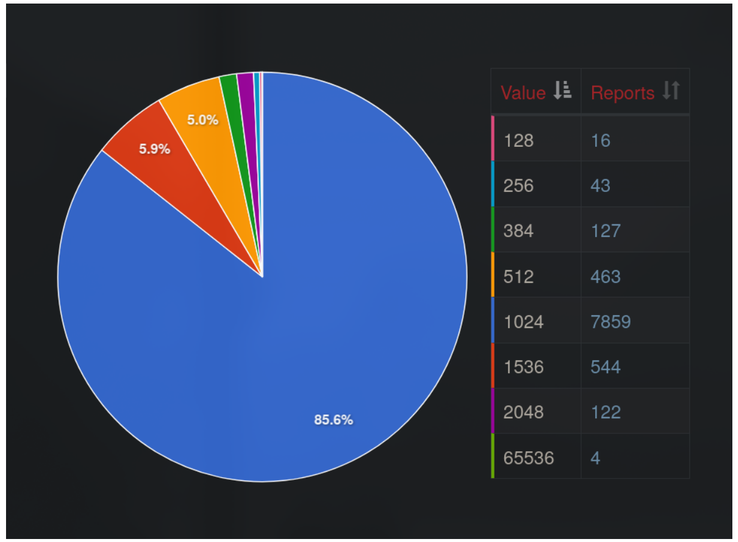
Work Group Device Limits

maxComputeWorkGroupCount[0]



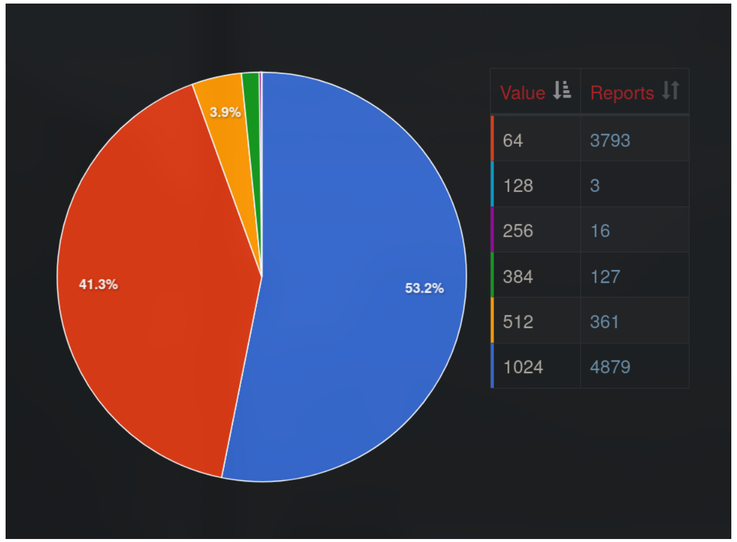
Value	Reports
65535	2831
65536	537
1073741824	580
2147483647	3164
4294967295	2067

maxComputeWorkGroupInvocations



Value	Reports
128	16
256	43
384	127
512	463
1024	7859
1536	544
2048	122
65536	4

maxComputeWorkGroupSize[2]



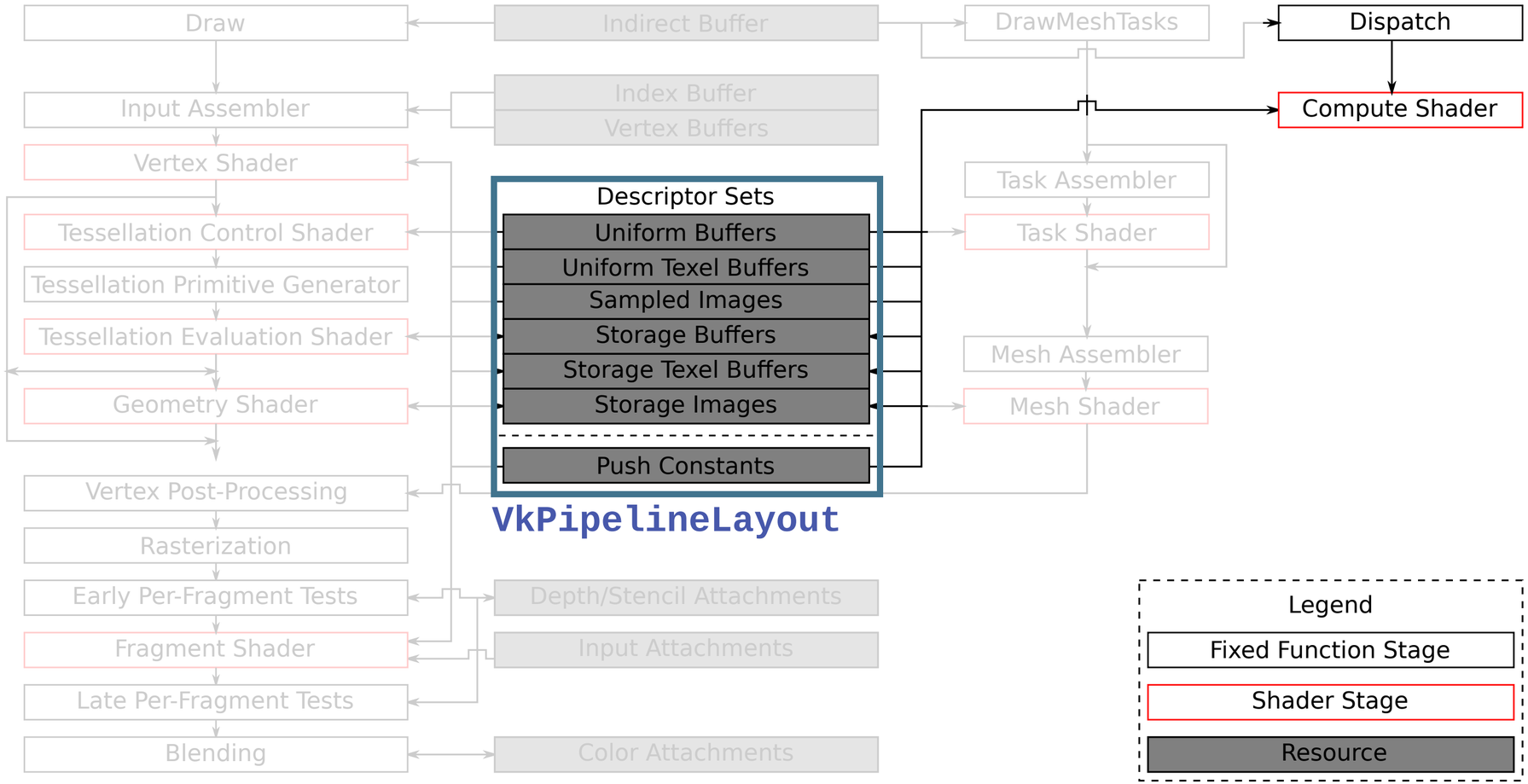
Value	Reports
64	3793
128	3
256	16
384	127
512	361
1024	4879

https://vulkan.gpuinfo.org/listpropertiescore10.php

Recording a Command Buffer

```
1  void recordCommandBuffer(VkCommandBuffer commandBuffer) {
2      vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_COMPUTE, m_pipeline);
3
4      vkCmdDispatch(commandBuffer, div_ceil(NUM_PARTICLES, 256), 1, 1);
5
6      ...
7
8
9
10
11
12
13
14
15
16 }
```

Vulkan Pipelines Overview



https://docs.vulkan.org/spec/latest/chapters/pipelines.html

Pipeline Layouts

```
1  VkResult vkCreatePipelineLayout(
2      VkDevice                                    device,
3      const VkPipelineLayoutCreateInfo*           pCreateInfo,
4      const VkAllocationCallbacks*                pAllocator,
5      VkPipelineLayout*                           pPipelineLayout);
```

```
1  typedef struct VkPipelineLayoutCreateInfo {
2      VkStructureType                 sType;
3      const void*                     pNext;
4      VkPipelineLayoutCreateFlags     flags;
5      uint32_t                        setLayoutCount;
6      const VkDescriptorSetLayout*    pSetLayouts;
7      uint32_t                        pushConstantRangeCount;
8      const VkPushConstantRange*      pPushConstantRanges;
9  } VkPipelineLayoutCreateInfo;
```

- Pipeline layouts define what resources the shaders can use.
- Zero or more descriptor set layouts and zero or more push constant ranges can be defined.

Push Constants

```
layout(push_constant) uniform PushConstants {
    float deltaTime;
    uint  particleCount;
} pc;
```

```
struct ComputePushConstants {
    float    deltaTime;
    uint32_t particleCount;
};

const VkPushConstantRange pushConstant{
    .stageFlags = VK_SHADER_STAGE_COMPUTE_BIT,
    .offset = 0,
    .size = sizeof(ComputePushConstants),
};
```

- Push constants are small banks of uniform values available to shaders without creating buffers or descriptor sets.
- Multiple push constant ranges for different stages can be defined.

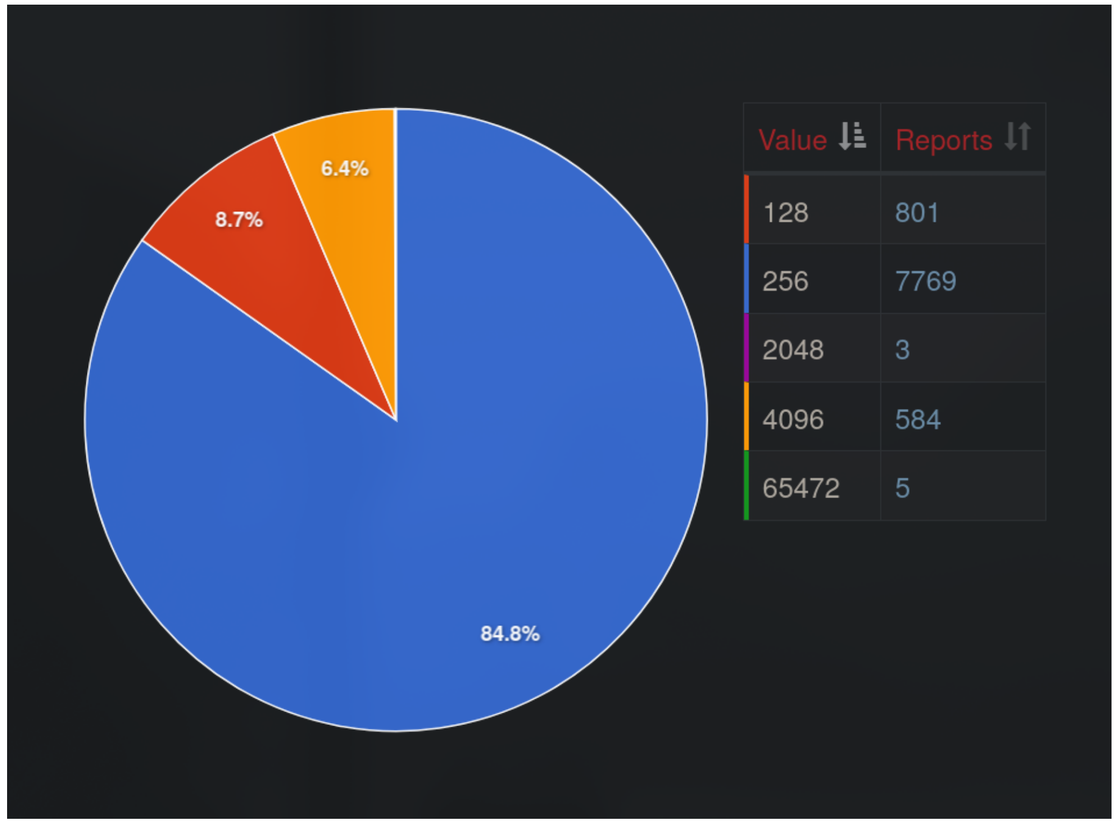
Push Constants

```
const ComputePushConstants pc{
    .deltaTime = frameTime,
    .particleCount = NUM_PARTICLES,
};

vkCmdPushConstants(
    commandBuffer,
    m_pipelineLayout,
    VK_SHADER_STAGE_COMPUTE_BIT,
    0,                              // offset
    sizeof(pc),                     // size
    &pc
);
```

- Copied into the command buffer
- Each call overwrites only the specified byte range
- Ideal for small data that is different for every draw call (e.g. model matrix)
- Heavily limited in size: only 128 bytes guaranteed up to Vulkan 1.3

Push Constants

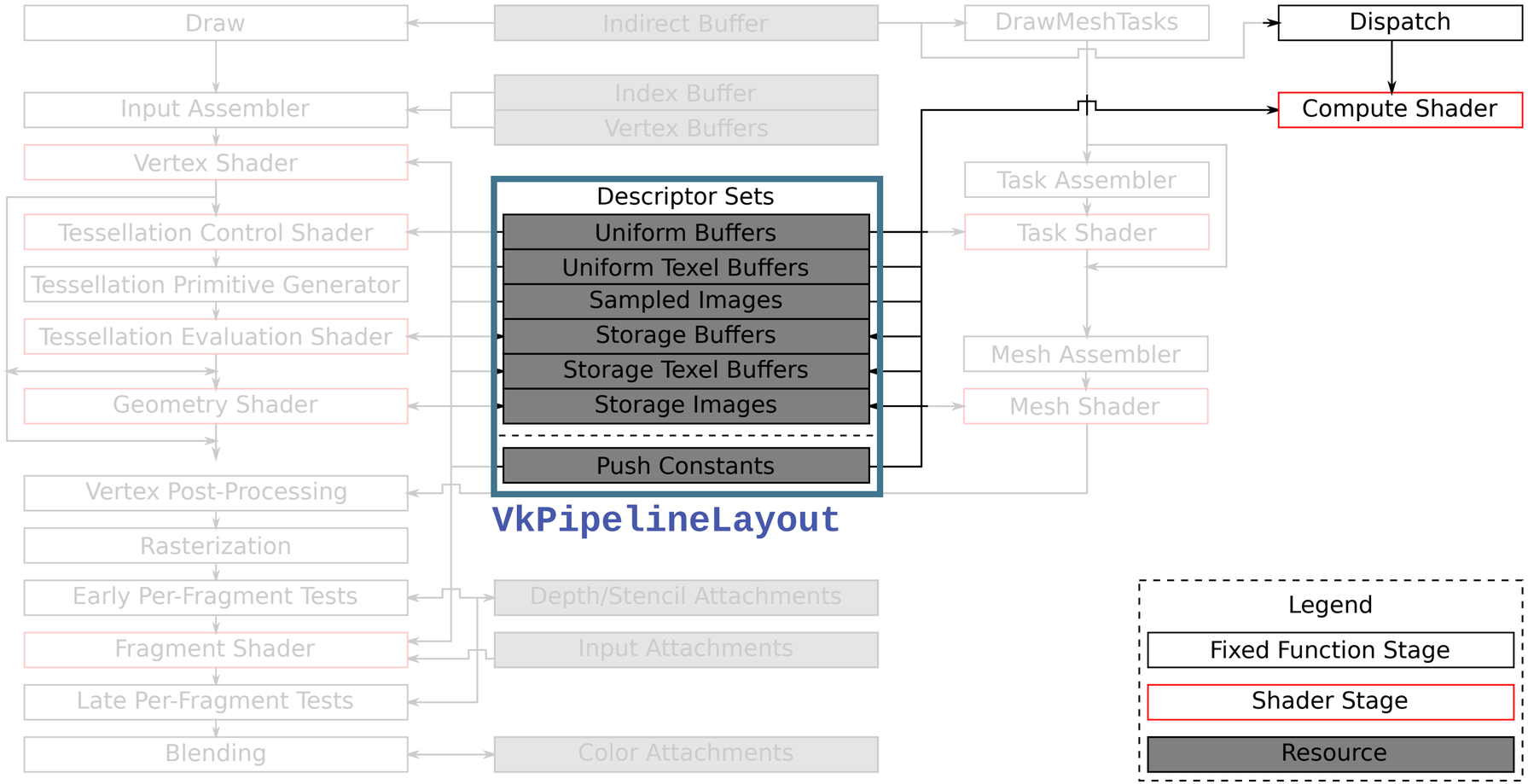


Value	Reports
128	801
256	7769
2048	3
4096	584
65472	5

Keep maximum size in mind!

- 128 bytes guaranteed by Vulkan spec (256 bytes from Vulkan 1.4)
- 256 bytes supported by most desktop GPUs
- rarely larger, mostly when emulating Vulkan through MoltenVK on Apple devices

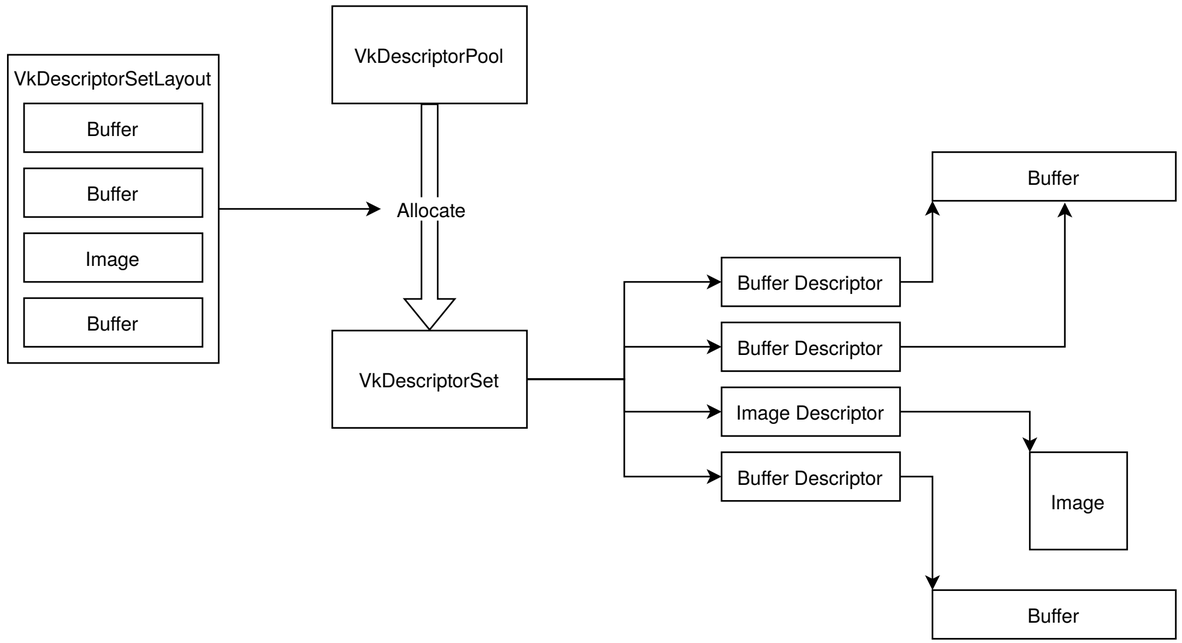
Vulkan Pipelines Overview



https://docs.vulkan.org/spec/latest/chapters/pipelines.html

Resource Descriptors

- Descriptors allow shaders to access resources, such as buffers, images, and samplers.



- Descriptor set layouts are the blueprints. They define a number of bindings.
- Descriptor pools manage the memory for sets and descriptors.
- Allocated descriptor sets contain descriptors according to the layout.
- The actual descriptors point to the resources, like buffers and images.
- Multiple descriptors can point to the same resources at different offsets.

Descriptor Bindings

```
1  layout (set = 1, binding = 0) uniform UBOFrame {
2      mat4 projection;
3      mat4 view;
4  } uboFrame;
5
6  layout (set = 2, binding = 0) uniform UBOMaterial {
7      float roughness;
8      float metallic;
9      float specular;
10 } uboMaterial;
11
12 // set = 0 is implicit
13 layout (binding = 0) uniform samplerCube samplerIrradiance;
14 layout (binding = 1) uniform sampler2D samplerBRDFLUT;
```

Descriptor Set Layout

```
VkResult vkCreateDescriptorSetLayout(
    VkDevice                                    device,
    const VkDescriptorSetLayoutCreateInfo*      pCreateInfo,
    const VkAllocationCallbacks*                pAllocator,
    VkDescriptorSetLayout*                      pSetLayout);
```

```
typedef struct VkDescriptorSetLayoutCreateInfo {
    VkStructureType                             sType;
    const void*                                 pNext;
    VkDescriptorSetLayoutCreateFlags            flags;
    uint32_t                                    bindingCount;
    const VkDescriptorSetLayoutBinding*         pBindings;
} VkDescriptorSetLayoutCreateInfo;
```

```
typedef struct VkDescriptorSetLayoutBinding {
    uint32_t                                    binding;
    VkDescriptorType                            descriptorType;
    uint32_t                                    descriptorCount;
    VkShaderStageFlags                          stageFlags;
    const VkSampler*                            pImmutableSamplers;
} VkDescriptorSetLayoutBinding;
```

- `binding` is the same index as defined in the shader.
- `VkDescriptorType` defines if the descriptor points to a buffer, image, ...
- `descriptorCount` is used to create arrays of descriptor at one binding.
- `stageFlags` can specify multiple stages where the descriptor is available.
- `pImmutableSamplers` is used for performance optimization of image samplers.

Descriptor Types

```
typedef enum VkDescriptorType {
    VK_DESCRIPTOR_TYPE_SAMPLER = 0,
    VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER = 1,
    VK_DESCRIPTOR_TYPE_SAMPLED_IMAGE = 2,
    VK_DESCRIPTOR_TYPE_STORAGE_IMAGE = 3,
    VK_DESCRIPTOR_TYPE_UNIFORM_TEXEL_BUFFER = 4,
    VK_DESCRIPTOR_TYPE_STORAGE_TEXEL_BUFFER = 5,
    VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER = 6,
    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER = 7,
    VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC = 8,
    VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC = 9,
    VK_DESCRIPTOR_TYPE_INPUT_ATTACHMENT = 10,
    // Provided by VK_VERSION_1_3
    VK_DESCRIPTOR_TYPE_INLINE_UNIFORM_BLOCK = 1000138000,
    ...
} VkDescriptorType;
```

- `UNIFORM_BUFFER`: fast read-only access, 16kB guaranteed, 64kB is a safe limit on most GPUs
- `COMBINED_IMAGE_SAMPLER`: image and sampler in one descriptor. Equivalent to OpenGL texture
- `SAMPLER` + `SAMPLED_IMAGE`: separate descriptors for sampler reuse or different samplers for one image
- `STORAGE_BUFFER`, `STORAGE_IMAGE`: read-write access

Descriptor Set Layout Creation

```
1  VkDescriptorSetLayoutBinding set0[] = {
2    { 0, VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER,         1, VK_SHADER_STAGE_VERTEX_BIT },
3    { 1, VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, 1, VK_SHADER_STAGE_FRAGMENT_BIT },
4    { 2, VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, 1, VK_SHADER_STAGE_FRAGMENT_BIT },
5  };
6
7  VkDescriptorSetLayoutCreateInfo info {
8      .sType        = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO,
9      .bindingCount = sizeof(set0) / sizeof(set0[0]),
10     .pBindings    = set0,
11 };
12
13 VK_CHECK(vkCreateDescriptorSetLayout(device, &info, nullptr, &m_set0Layout));
```

- The set index is implicitly defined by the array index in the pipeline layout.

Descriptor Pool

```
1 VkResult vkCreateDescriptorPool(
2     VkDevice                                device,
3     const VkDescriptorPoolCreateInfo*       pCreateInfo,
4     const VkAllocationCallbacks*            pAllocator,
5     VkDescriptorPool*                       pDescriptorPool);
```

```
1 typedef struct VkDescriptorPoolCreateInfo {
2     VkStructureType                         sType;
3     const void*                             pNext;
4     VkDescriptorPoolCreateFlags             flags;
5     uint32_t                                maxSets;
6     uint32_t                                poolSizeCount;
7     const VkDescriptorPoolSize*             pPoolSizes;
8 } VkDescriptorPoolCreateInfo;
```

```
1 typedef struct VkDescriptorPoolSize {
2     VkDescriptorType                        type;
3     uint32_t                                descriptorCount;
4 } VkDescriptorPoolSize;
```

- `maxSets` is the number of `VkDescriptorSet` in the pool.
- Each `VkDescriptorPoolSize` sets the number of descriptors of one specific type in the pool.

Descriptor Pool Example

```
1  const VkDescriptorPoolSize poolSizes[] = {
2      { .type = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER, .descriptorCount = framesInFlight * 2 },
3      { .type = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, .descriptorCount = framesInFlight * 4 },
4      { .type = VK_DESCRIPTOR_TYPE_STORAGE_BUFFER, .descriptorCount = framesInFlight },
5  };
6
7  const VkDescriptorPoolCreateInfo descriptorPoolInfo{
8      .sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO,
9      .maxSets = framesInFlight * 2,
10     .poolSizeCount = sizeof(poolSizes) / sizeof(poolSizes[0]),
11     .pPoolSizes = poolSizes,
12 };
13
14 VK_CHECK(vkCreateDescriptorPool(device, &descriptorPoolInfo, nullptr, &descriptorPool));
```

- Two descriptor sets per frame.
- The first contains one uniform and one storage buffer.
- The second contains one uniform buffer and four textures.

Allocating Descriptor Sets

```
1 VkResult vkAllocateDescriptorSets(
2     VkDevice                                    device,
3     const VkDescriptorSetAllocateInfo*          pAllocateInfo,
4     VkDescriptorSet*                            pDescriptorSets);
```

```
1 typedef struct VkDescriptorSetAllocateInfo {
2     VkStructureType                             sType;
3     const void*                                 pNext;
4     VkDescriptorPool                            descriptorPool;
5     uint32_t                                    descriptorSetCount;
6     const VkDescriptorSetLayout*                pSetLayouts;
7 } VkDescriptorSetAllocateInfo;
```

- Allocates a number of descriptor sets according to a descriptor set layout from the pool.
- Allocation fails if the pool is out of descriptor sets or descriptors of a required type.
- Freeing single descriptor sets is possible via `vkFreeDescriptorSets`, but is not recommended (requires `VK_DESCRIPTOR_POOL_CREATE_FREE_DESCRIPTOR_SET_BIT`).
- Instead, if necessary, reset the entire pool (`vkResetDescriptorPool`).

Updating Descriptor Sets

```
1  void vkUpdateDescriptorSets(
2      VkDevice                                    device,
3      uint32_t                                    descriptorWriteCount,
4      const VkWriteDescriptorSet*                 pDescriptorWrites,
5      uint32_t                                    descriptorCopyCount,
6      const VkCopyDescriptorSet*                  pDescriptorCopies);
```

```
1  typedef struct VkWriteDescriptorSet {
2      VkStructureType                             sType;
3      const void*                                 pNext;
4      VkDescriptorSet                             dstSet;
5      uint32_t                                    dstBinding;
6      uint32_t                                    dstArrayElement;
7      uint32_t                                    descriptorCount;
8      VkDescriptorType                            descriptorType;
9      const VkDescriptorImageInfo*                pImageInfo;
10     const VkDescriptorBufferInfo*               pBufferInfo;
11     const VkBufferView*                         pTexelBufferView;
12 } VkWriteDescriptorSet;
```

- `vkUpdateDescriptorSets` takes any number of descriptor writes that are performed in order.
- The descriptor write takes the destination set, binding and again the type of the descriptor.
- Either `pImageInfo` or `pBufferInfo` is specified, depending on if images/samplers or buffers are written. Texel buffers use `pTexelBufferView` and other descriptors use `pNext`.

Updating Descriptor Sets

```
typedef struct VkDescriptorImageInfo {
    VkSampler       sampler;
    VkImageView     imageView;
    VkImageLayout   imageLayout;
} VkDescriptorImageInfo;
```

```
typedef struct VkDescriptorBufferInfo {
    VkBuffer        buffer;
    VkDeviceSize    offset;
    VkDeviceSize    range;
} VkDescriptorBufferInfo;
```

Binding Descriptor Sets

```
1  void recordCommandBuffer(VkCommandBuffer commandBuffer) {
2      vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_COMPUTE, m_pipeline);
3
4      vkCmdBindDescriptorSets(
5          commandBuffer,
6          VK_PIPELINE_BIND_POINT_COMPUTE,
7          m_pipelineLayout,
8          0,                                  // firstSet
9          1,                                  // descriptorSetCount
10         &m_descriptorSets[currentFrame],
11         0,                                  // dynamicOffsetCount
12         nullptr                             // pDynamicOffsets
13     );
14
15     vkCmdDispatch(commandBuffer, div_ceil(NUM_PARTICLES, 256), 1, 1);
16 }
```

Descriptor Set Grouping

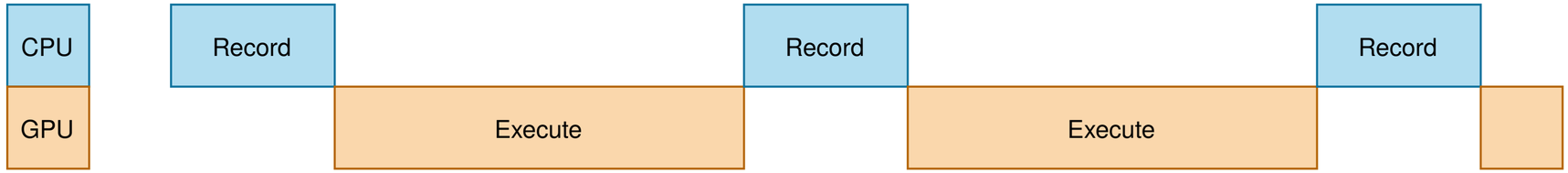
- *Why have more than one descriptor set?*
  Can we not just put all descriptors for one pipeline into a single gigantic set?

- Our data changes at different frequencies (scene, frame, material, object).
  Worst case: `n_objects × n_frames_in_flight` large descriptor sets.
  Everything has to be rebound for every draw call.

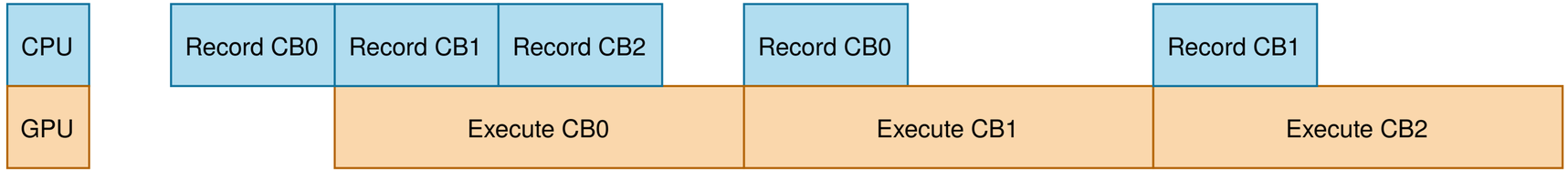
- Instead: group descriptors into sets by update frequency.
  - Set 0: per-scene data (shadow map, environment map, …)
  - Set 1: per-frame data (view/projection matrix, time, …)
  - Set 2: per-material data (roughness, metallic, textures, …)
  - Set 3: per-object data (model matrix) → could be push constants
- Device limits: many mobile GPUs support only 4 sets, desktop 8–32.

Frames in Flight

- Command buffers may not be written to again, while the GPU is still using them.
- Single command buffer (‘one frame in flight’):

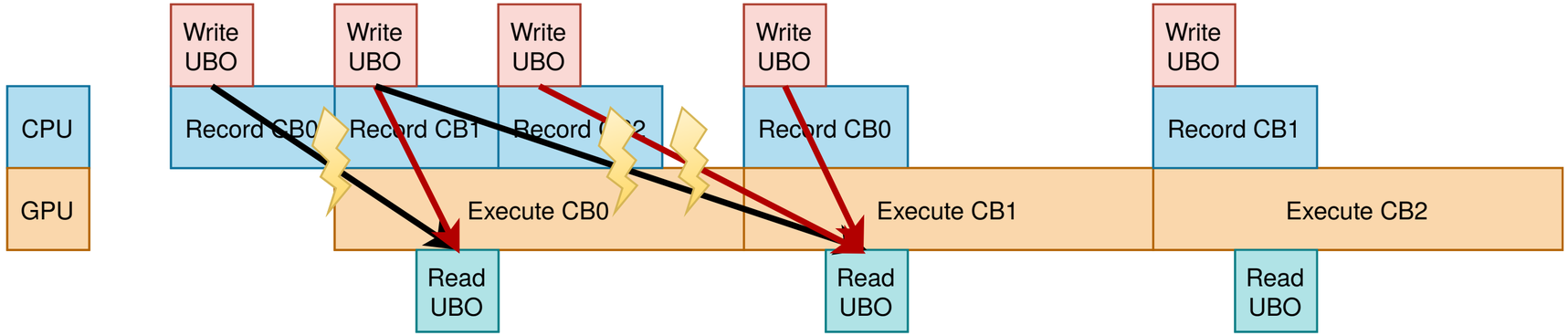


- Pre-record multiple command buffers (‘multiple frames in flight’):



Frames in Flight

- Problem: CPU overwrites data before GPU can read it.
- Example: Uniform buffer with view matrix that changes every frame:



- Solution: Create `n_frames_in_flight` instances of every buffer that is written to by the CPU.
- Cycle through the buffers and their descriptors when writing the command buffers.
- Requires one descriptor set per in-flight frame.

Recap - Setting up Compute

- Shader modules contain SPIR-V code
- Shader stage info adds stage flag, entry point and specialization constants
- Pipeline layout defines what resources the shader can use
- Push constants for small, frequently changing data
- Descriptor sets for buffers and images, grouped by update frequency
- Descriptor sets are allocated from descriptor pools using descriptor set layouts
- Buffers and images are written to the descriptors of the descriptor sets
- Compute pipelines are created from shader stage and pipeline layout
- Recording: bind pipeline and descriptor sets, push constants, then dispatch

⇒ **Next:** *How do we create some buffers now?*

Buffers & Memory

Buffers

- Buffers are linear arrays of data to be used in a variety of ways.

```
VkResult vkCreateBuffer(
    VkDevice                        device,
    const VkBufferCreateInfo*       pCreateInfo,
    const VkAllocationCallbacks*    pAllocator,
    VkBuffer*                       pBuffer);
```

```
typedef struct VkBufferCreateInfo {
    VkStructureType                 sType;
    const void*                     pNext;
    VkBufferCreateFlags             flags;
    VkDeviceSize                    size;
    VkBufferUsageFlags              usage;
    VkSharingMode                   sharingMode;
    uint32_t                        queueFamilyIndexCount;
    const uint32_t*                 pQueueFamilyIndices;
} VkBufferCreateInfo;
```

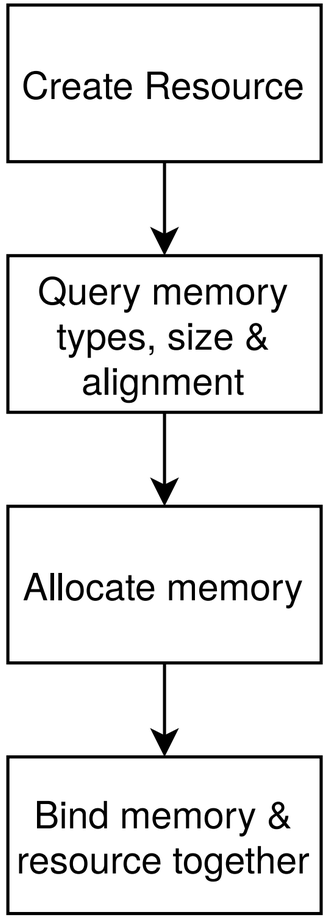
- `size` : size of buffer in bytes
- `usage` : tells the driver what the buffer will be used for
- `sharingMode` : defines if the buffer will be used by more than one queue family
- if `VK_SHARING_MODE_CONCURRENT`, queue families that will access the buffer have to be specified through `pQueueFamilyIndices`

Buffer Usage

```
typedef enum VkBufferUsageFlagBits {
    VK_BUFFER_USAGE_TRANSFER_SRC_BIT = 0x00000001,
    VK_BUFFER_USAGE_TRANSFER_DST_BIT = 0x00000002,
    VK_BUFFER_USAGE_UNIFORM_TEXEL_BUFFER_BIT = 0x00000004,
    VK_BUFFER_USAGE_STORAGE_TEXEL_BUFFER_BIT = 0x00000008,
    VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT = 0x00000010,
    VK_BUFFER_USAGE_STORAGE_BUFFER_BIT = 0x00000020,
    VK_BUFFER_USAGE_INDEX_BUFFER_BIT = 0x00000040,
    VK_BUFFER_USAGE_VERTEX_BUFFER_BIT = 0x00000080,
    VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT = 0x00000100,
    // Provided by VK_VERSION_1_2
    VK_BUFFER_USAGE_SHADER_DEVICE_ADDRESS_BIT = 0x00020000,
    // Provided by VK_KHR_video_decode_queue
    VK_BUFFER_USAGE_VIDEO_DECODE_SRC_BIT_KHR = 0x00002000,
    // Provided by VK_KHR_video_decode_queue
    VK_BUFFER_USAGE_VIDEO_DECODE_DST_BIT_KHR = 0x00004000,
  ...
} VkBufferUsageFlagBits;
```

Buffer Memory

- Creating a buffer does not automatically allocate memory.
- Remember: *memory allocation is manual in Vulkan*.
- Memory is allocated after resource creation and explicitly bound to the resources.
- You know your application better than the driver.
  - Achieve smaller footprint
  - Optimize for specific platforms
  - Make use of aliasing



Allocating Memory

```
1  VkResult vkAllocateMemory(
2      VkDevice                        device,
3      const VkMemoryAllocateInfo*     pAllocateInfo,
4      const VkAllocationCallbacks*    pAllocator,
5      VkDeviceMemory*                 pMemory);
```

```
1  typedef struct VkMemoryAllocateInfo {
2      VkStructureType                 sType;
3      const void*                     pNext;
4      VkDeviceSize                    allocationSize;
5      uint32_t                        memoryTypeIndex;
6  } VkMemoryAllocateInfo;
```

- Allocations are guaranteed to meet any alignment requirement of the implementation.
  If the highest alignment requirement is 128 bytes, all allocations will be aligned to 128 bytes.

- *How do we find out the correct memory type index?*

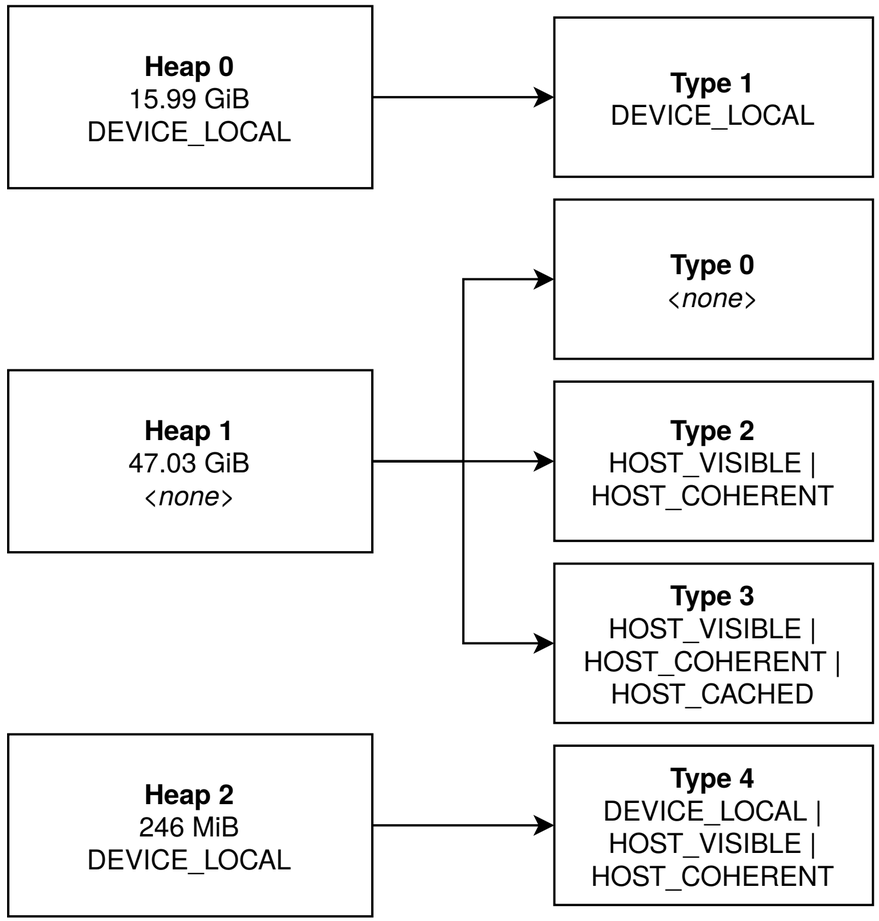
Memory Types

```
1  void vkGetPhysicalDeviceMemoryProperties(
2      VkPhysicalDevice                    physicalDevice,
3      VkPhysicalDeviceMemoryProperties*   pMemoryProperties);
```

```
1  typedef struct VkPhysicalDeviceMemoryProperties {
2      uint32_t                            memoryTypeCount;
3      VkMemoryType                        memoryTypes[VK_MAX_MEMORY_TYPES];
4      uint32_t                            memoryHeapCount;
5      VkMemoryHeap                        memoryHeaps[VK_MAX_MEMORY_HEAPS];
6  } VkPhysicalDeviceMemoryProperties;
```

```
1  typedef struct VkMemoryType {
2      VkMemoryPropertyFlags               propertyFlags;
3      uint32_t                            heapIndex;
4  } VkMemoryType;
5
6  typedef struct VkMemoryHeap {
7      VkDeviceSize                        size;
8      VkMemoryHeapFlags                   flags;
9  } VkMemoryHeap;
```

Memory Types - Desktop RTX 4080



on-device VRAM, fast read and write access, inaccessible to CPU

system memory, slow GPU access, GPU only

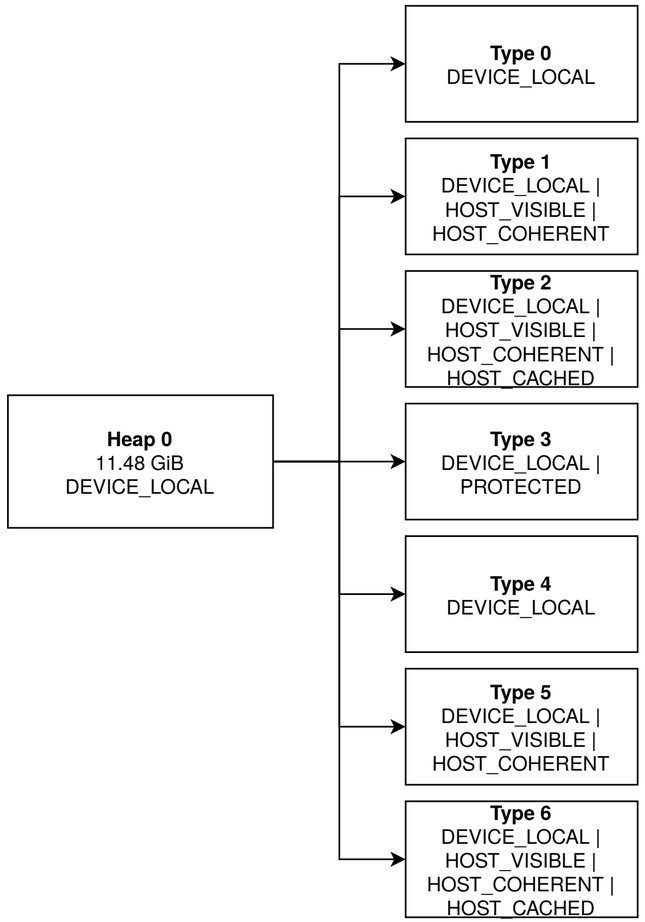
system memory, slow GPU access, fast CPU write

system memory, slow GPU access, fast CPU read/write

on-device VRAM, fast access, fast CPU write / slow read

Note: With Resizable BAR, the entire range of device-local memory may be host-visible.

Memory Types - Integrated i7-12700H



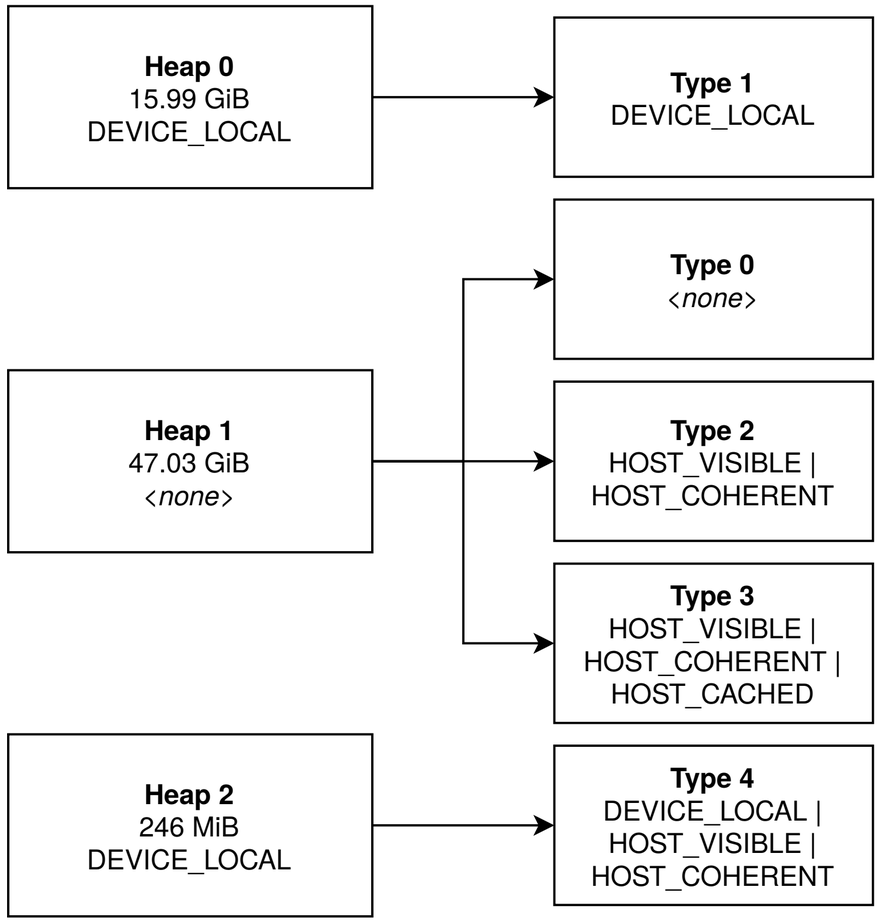
- everything is system memory, everything is ‘device local’
- types differ in how they are mapped to CPU address space
- generally: if property flags are equal, memory types are sorted in order of performance
- not all resources are compatible with all memory types

Memory Properties

```
typedef enum VkMemoryPropertyFlagBits {
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT = 0x00000001,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT = 0x00000002,
    VK_MEMORY_PROPERTY_HOST_COHERENT_BIT = 0x00000004,
    VK_MEMORY_PROPERTY_HOST_CACHED_BIT = 0x00000008,
    VK_MEMORY_PROPERTY_LAZILY_ALLOCATED_BIT = 0x00000010,
    // Provided by VK_VERSION_1_1
    VK_MEMORY_PROPERTY_PROTECTED_BIT = 0x00000020,
    // Provided by VK_AMD_device_coherent_memory
    VK_MEMORY_PROPERTY_DEVICE_COHERENT_BIT_AMD = 0x00000040,
    // Provided by VK_AMD_device_coherent_memory
    VK_MEMORY_PROPERTY_DEVICE_UNCACHED_BIT_AMD = 0x00000080,
    // Provided by VK_NV_external_memory_rdma
    VK_MEMORY_PROPERTY_RDMA_CAPABLE_BIT_NV = 0x00000100,
} VkMemoryPropertyFlagBits;
```

- `DEVICE_LOCAL`: most efficient for device access
- `HOST_VISIBLE`: memory can be mapped for host access
- `HOST_COHERENT`: host cache management commands are not needed, writes automatically propagate to device memory
- `HOST_CACHED`: memory is cached on the host, reads to uncached memory are slow

Memory Types - Preferred Usage



‘upload-once’ resources, textures, render targets, depth buffers, storage buffers

VRAM overflow

staging buffers for transfer to device local memory

data written by GPU and read by CPU

dynamic, write once, read once resources

Buffer Memory Allocation

```
1  const VkBufferCreateInfo bufferInfo{
2      .sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,
3      .size = bufferSize,
4      .usage = bufferUsage,
5      .sharingMode = VK_SHARING_MODE_EXCLUSIVE,
6  };
7  VK_CHECK(vkCreateBuffer(device, &bufferInfo, nullptr, &buffer));
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
```

Buffer Memory Requirements

```
1 void vkGetBufferMemoryRequirements(
2     VkDevice                    device,
3     VkBuffer                    buffer,
4     VkMemoryRequirements*       pMemoryRequirements);
```

```
1 typedef struct VkMemoryRequirements {
2     VkDeviceSize                size;
3     VkDeviceSize                alignment;
4     uint32_t                    memoryTypeBits;
5 } VkMemoryRequirements;
```

- `size`: the size in bytes required for the allocation (may be larger than the size of the buffer)
- `alignment`: the alignment in bytes required for the allocation
- `memoryTypeBits`: a bitmask where bits are set when the corresponding memory type is supported for the resource. (bit `i` set → type `i` supported)

Buffer Memory Allocation

```
1  const VkBufferCreateInfo bufferInfo{
2      .sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,
3      .size = bufferSize,
4      .usage = bufferUsage,
5      .sharingMode = VK_SHARING_MODE_EXCLUSIVE,
6  };
7  VK_CHECK(vkCreateBuffer(device, &bufferInfo, nullptr, &buffer));
8
9  VkMemoryRequirements memRequirements;
10 vkGetBufferMemoryRequirements(device, buffer, &memRequirements);
11
12 // find first memory type that has its bit set in memoryTypeBits and has the required properties
13 const uint32_t memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, propertyFlags);
14
15 const VkMemoryAllocateInfo allocInfo{
16     .sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO,
17     .allocationSize = memRequirements.size,
18     .memoryTypeIndex = memoryTypeIndex,
19 };
20 VK_CHECK(vkAllocateMemory(device, &allocInfo, nullptr, &memory));
21
22 VK_CHECK(vkBindBufferMemory(device, buffer, memory, 0));
```

Memory Allocation Strategy

- Memory allocations are **slow**. Perform all allocations during loading times and avoid allocations in between frames.
- Upload to device-local memory using staging buffers. Allocate one buffer in device-local memory and one buffer in host-visible memory, then transfer the data.
- Frequently updated uniform buffers can sometimes stay in non-device memory. Automatic caching by the GPU may compensate. Perform tests to verify.
- Never hardcode memory type indices. They are magic numbers that change between devices and driver versions.

Over-subscription

- *What happens when you run out of video memory?*
- Depending on the driver, allocations may
  - fail with `VK_ERROR_OUT_OF_DEVICE_MEMORY`.
  - succeed and some blocks are silently migrated to system memory.
- Why migrate? So applications don’t crash as soon as VRAM is full.
  - Especially useful during development.
  - There are other applications running on the machine that need memory.
- Other things need memory too: descriptors, command buffers, swap chains, …
- Extensions (`VK_EXT_memory_budget`) can estimate available memory.
- Recommended by the Khronos Group: Use no more than
  - 80% of `DEVICE_LOCAL` memory.
  - 66% of `DEVICE_LOCAL|HOST_VISIBLE` memory.

Memory Mapping

```
1 VkResult vkMapMemory(
2     VkDevice                device,
3     VkDeviceMemory          memory,
4     VkDeviceSize            offset,
5     VkDeviceSize            size,
6     VkMemoryMapFlags        flags,
7     void**                  ppData);
```

- `vkMapMemory` retrieves a host virtual address pointer to part of or a whole allocation.
- It is fine to keep an entire object mapped for its entire lifetime.
- Mapping the same allocation twice is considered an error.
- If `VK_MEMORY_PROPERTY_HOST_COHERENT_BIT` is set, writes are automatically propagated to device memory. Otherwise `vkFlushMappedMemoryRanges` and `vkInvalidateMappedMemoryRanges` are needed for manual cache control.
- On submit, all host coherent writes are resolved before command buffer begins executing.
- Memory objects can be unmapped using `vkUnmapMemory`.

Staging Buffers

```
1  void vkCmdCopyBuffer(
2      VkCommandBuffer        commandBuffer,
3      VkBuffer               srcBuffer,
4      VkBuffer               dstBuffer,
5      uint32_t               regionCount,
6      const VkBufferCopy*    pRegions);
```

```
1  typedef struct VkBufferCopy {
2      VkDeviceSize           srcOffset;
3      VkDeviceSize           dstOffset;
4      VkDeviceSize           size;
5  } VkBufferCopy;
```

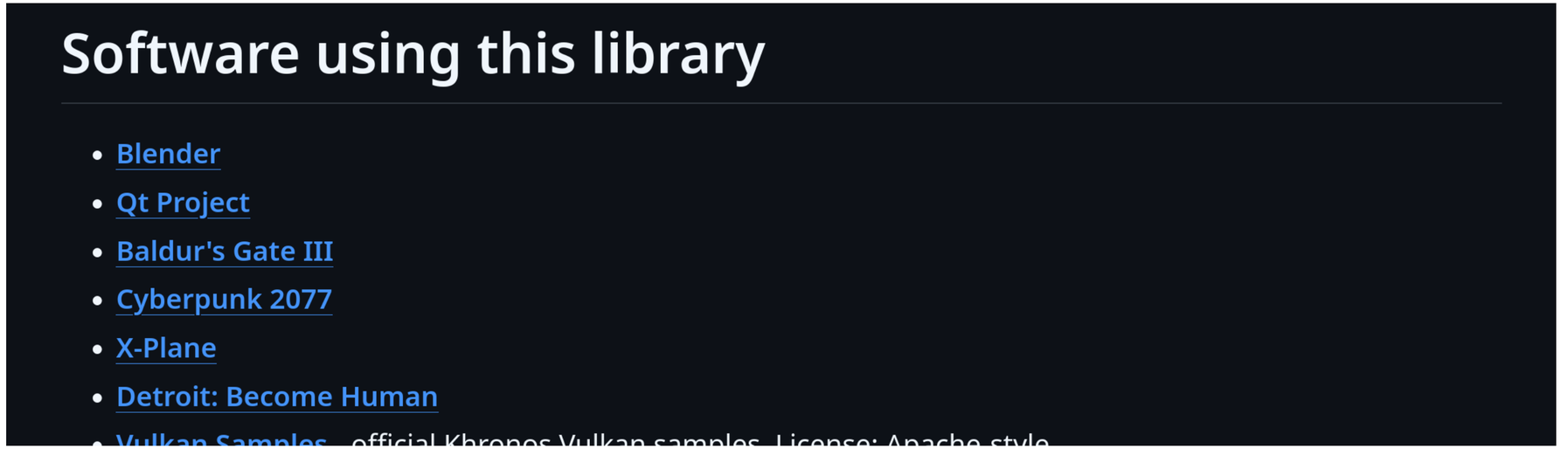
- First, create a host-visible staging buffer, map it to CPU memory and copy the data to that buffer.
- Then, create a device-local buffer and transfer the data using `vkCmdCopyBuffer`.
- Remember: the command is recorded into a command buffer and is an asynchronous operation. The staging buffer must be valid at least until the transfer is complete.
- Afterwards, the staging buffer can be destroyed.

Sub-allocation

- *On the CPU, would you allocate a separate memory page for every heap object?*
- Bad for performance, data locality, fragmentation, …
- Also: maximum number of allocations is limited (`maxMemoryAllocationCount`).
  Only 4096 guaranteed, in practice on most desktop devices much higher.
- Instead: allocate larger chunks from the driver and sub-allocate your buffers and images.
- Choose any common memory allocation strategy to slice blocks into sub-allocations.
- Keep alignment in mind.

Vulkan Memory Allocator

- https://github.com/GPUOpen-LibrariesAndSDKs/VulkanMemoryAllocator
- Single header, open source library by AMD
- Similar interface to core Vulkan
- Helps you choose correct and optimal memory type based on intended usage.
- Automatically sub-allocates blocks, finds optimal slots in unused ranges, ...
- Supports multi-threading, memory mapping, non-coherent memory, ...



Vulkan Memory Allocator

```
1  const VkBufferCreateInfo bufferInfo = {
2      .sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,
3      .size = 65536,
4      .usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT,
5  };
6
7  const VmaAllocationCreateInfo allocInfo = {
8      .usage = VMA_MEMORY_USAGE_AUTO,
9  };
10
11 VkBuffer buffer;
12 VmaAllocation allocation;
13 vmaCreateBuffer(allocator, &bufferInfo, &allocInfo, &buffer, &allocation, nullptr);
```

Vulkan Memory Allocator

```
typedef struct VmaAllocationCreateInfo {
    VmaAllocationCreateFlags    flags;
    VmaMemoryUsage              usage;
    VkMemoryPropertyFlags       requiredFlags;
    VkMemoryPropertyFlags       preferredFlags;
    uint32_t                    memoryTypeBits;
    VmaPool VMA_NULLABLE        pool;
    void* VMA_NULLABLE          pUserData;
    float                       priority;
    VkDeviceSize                minAlignment;
} VmaAllocationCreateInfo;
```

```
typedef enum VmaMemoryUsage {
    VMA_MEMORY_USAGE_UNKNOWN    = 0,
    VMA_MEMORY_USAGE_GPU_ONLY   = 1, // deprecated
    VMA_MEMORY_USAGE_CPU_ONLY   = 2, // deprecated
    VMA_MEMORY_USAGE_CPU_TO_GPU = 3, // deprecated
    VMA_MEMORY_USAGE_GPU_TO_CPU = 4, // deprecated
    VMA_MEMORY_USAGE_CPU_COPY   = 5, // deprecated
    VMA_MEMORY_USAGE_GPU_LAZILY_ALLOCATED = 6,
    VMA_MEMORY_USAGE_AUTO                 = 7,
    VMA_MEMORY_USAGE_AUTO_PREFER_DEVICE   = 8,
    VMA_MEMORY_USAGE_AUTO_PREFER_HOST     = 9,
} VmaMemoryUsage;
```

```
typedef enum VmaAllocationCreateFlagBits {
    VMA_ALLOCATION_CREATE_DEDICATED_MEMORY_BIT              = 0x00000001,
    VMA_ALLOCATION_CREATE_NEVER_ALLOCATE_BIT                = 0x00000002,
    VMA_ALLOCATION_CREATE_MAPPED_BIT                        = 0x00000004,
    ...
    VMA_ALLOCATION_CREATE_HOST_ACCESS_SEQUENTIAL_WRITE_BIT  = 0x00000400,
    VMA_ALLOCATION_CREATE_HOST_ACCESS_RANDOM_BIT            = 0x00000800,
    ...
} VmaAllocationCreateFlagBits;
```

Recap - Buffers & Memory

- Buffers are created with a specific size and usage flags.
- Resources do not immediately come with allocated memory.
- Drivers provide multiple types of memory from memory heaps.
- Fast device-local GPU-only memory is optimal for most resources.
- Host-visible memory is used for uploads from the CPU.
- Not all memory types support all resources.
- Some drivers silently migrate to system memory when VRAM is full.
- Sub-allocate multiple resources from larger blocks of memory.
- Use the Vulkan Memory Allocator.

Next Time

- Images, the Swap-Chain and Synchronization